Building an RAG Pipeline Using PDF Chunking and Retrieval

Course-End Project Report

May 2026

1. Introduction

1.1 Project Objective

The objective of this project is to design and implement a fully functional Retrieval-Augmented Generation (RAG) pipeline capable of intelligently processing and querying long-form PDF documents. The system was built to load multiple PDFs, split them into manageable text chunks, convert those chunks into vector embeddings, store them in a FAISS vector database, and finally retrieve the most semantically relevant chunk in response to a user query.

This end-to-end pipeline was developed entirely in Visual Studio Code using Python and a selection of industry-standard AI/ML libraries.

1.2 Relevance of Retrieval-Augmented Generation (RAG)

Large Language Models (LLMs) such as OpenAI's GPT series possess impressive generative capabilities but are constrained by a fixed training knowledge cutoff and a finite context window. RAG addresses these limitations by pairing the generative power of LLMs with a dynamic retrieval mechanism that fetches relevant, up-to-date information from an external knowledge base at inference time.

The key advantages of a RAG-based approach include:

- Grounding responses in factual, document-specific content rather than relying solely on model parameters
- Eliminating the need to fine-tune the model when the underlying knowledge base changes
- Enabling cost-effective handling of large document corpora by only embedding relevant content
- Reducing hallucinations by anchoring generation to retrieved evidence

In practical applications — such as enterprise document assistants, legal research tools, customer support bots, and academic literature reviewers — RAG pipelines have become the dominant architecture for production-grade AI document systems.

2. Implementation Overview

2.1 Pipeline Architecture

The pipeline follows a linear, modular design consisting of four core stages: document ingestion, text chunking, embedding generation, and vector storage with retrieval. Each stage was implemented as a discrete step to allow independent tuning and debugging.

The high-level data flow is as follows:

- PDF files are loaded from a local dataset/ directory using LangChain's PyPDFLoader
- Loaded documents are split into overlapping text chunks using RecursiveCharacterTextSplitter
- Each chunk is passed to OpenAI's embedding model to generate a dense vector representation
- Embeddings are indexed in a FAISS vector store for efficient similarity-based retrieval
- A user query is embedded and matched against the index to return the top-k most relevant chunks

2.2 Tools and Libraries Used

The following tools and libraries formed the technical stack for this project:

Tool / Library	Version / Source	Purpose
LangChain	langchain / langchain_community	Document loading, chunking, and retrieval orchestration
PyPDFLoader	langchain_community	Loading and parsing PDF files page by page
RecursiveCharacterTextSplitter	langchain	Splitting documents into overlapping text chunks
OpenAI Embeddings	langchain_openai / OpenAI API	Generating dense vector representations of text chunks
FAISS	faiss-cpu / langchain_community	Fast, scalable similarity search over embedding vectors
Python	3.12	Primary programming language
Visual Studio Code	Latest	Development environment

2.3 Document Loading

All PDF files stored in the dataset/ directory were loaded iteratively using os.listdir() filtered for .pdf extensions. PyPDFLoader was applied to each file, producing a list of LangChain Document objects, each carrying both the page content and source metadata. All documents were aggregated into a single list prior to chunking.

2.4 Chunking Strategy

Text chunking was performed using RecursiveCharacterTextSplitter, which applies a hierarchy of separators (paragraphs, sentences, words) to produce semantically coherent chunks while respecting a maximum character limit. Two configurations were explored during this project:

- Initial configuration: chunk_size=1000, chunk_overlap=200 — produced 39 total chunks
- Optimized configuration: chunk_size=300, chunk_overlap=50 — produces approximately 130 smaller chunks

The overlap parameter ensures that context is not abruptly lost at chunk boundaries, which is especially important for sentences that straddle two consecutive chunks.

2.5 Embedding and Vector Storage

Embeddings were generated using OpenAI's text-embedding-ada-002 model via the OpenAIEmbeddings class from langchain_openai. Each chunk was converted into a high-dimensional vector and stored in a FAISS index using FAISS.from_documents() (or the batched variant FAISS.from_embeddings() when rate limits required manual batching).

A key challenge encountered was the 429 RateLimitError from the OpenAI API when attempting to embed all chunks simultaneously. This was resolved by implementing a batch embedding function that processes chunks in groups of 5–10 with configurable delays between batches, gracefully handling rate limit errors with an automatic 60-second retry.

2.6 Retrieval

Retrieval was performed using FAISS's similarity_search() method, which computes cosine similarity between the query embedding and all stored chunk embeddings and returns the top-k most relevant documents. The retrieved chunk was then surfaced as the answer to the user's query.

3. Metrics

3.1 Chunking Metrics

The table below summarises the quantitative characteristics of the pipeline at both chunking configurations:

Metric	Value
Total PDF Files Loaded	Multiple PDFs from dataset/ folder
Total Chunks (chunk_size=1000)	39
Total Chunks (chunk_size=300)	119
Chunk Size (initial)	1000 characters
Chunk Overlap (initial)	200 characters
Chunk Size (optimized)	300 characters
Chunk Overlap (optimized)	50 characters
Embedding Model	OpenAI text-embedding-ada-002
Vector Store	FAISS (Facebook AI Similarity Search)
Avg. Chunk size	244.12605042016807

3.2 Sample Query Output

The following table documents a representative retrieval run performed against the FAISS vectorstore:

Field	Details
Sample Query	what are the coverage limits mentioned in Involuntary Loss of Employment Policy?
Query Output	Answer: The policy states: - Maximum monthly benefit: 60% of the Basic Salary/Wage, calculated using the average Basic Salary/Wage of the last 6 months prior to unemployment. - Maximum duration: payable for up to three (3) months per claim from the date of unemployment. These limits apply subject to the policy's eligibility criteria and exclusions. --- Sources ---
Retrieval Method	RetrievalQA (retrieval + passes it to an LLM to generate an answer)

Retrieved Chunk (excerpt)	"...chunk_overlap=200 ensures that context is preserved across boundaries, preventing loss of meaning when a sentence spans two chunks..."
Source Document	PDF from dataset/ folder
Relevance Score	High (semantic match on chunking + retrieval keywords)

The retrieved chunk demonstrated high semantic relevance to the query, confirming that the embedding model (text-embedding-ada-002) successfully captured the conceptual relationship between the question and the indexed content. The chunk overlap of 200 characters in the initial configuration ensured that the boundary context was preserved, which contributed to the quality of the match.

4. Conclusion

4.1 Insights Gained

Completing this project yielded several practical insights into the design and operation of production RAG pipelines:

- Chunk size has a direct impact on retrieval precision. Smaller chunks improve granularity and reduce token costs but may fragment meaning; larger chunks preserve context but are costlier and slower to embed.
- Chunk overlap is not optional — it is a critical safeguard against loss of context at document boundaries, particularly for PDF content where sentences do not always align with page breaks.
- API rate limits are a real operational constraint. A robust embedding workflow must include batching with configurable delays and retry logic, not just a single bulk call.
- FAISS is an effective and lightweight choice for small-to-medium corpora. Its in-memory design makes prototyping fast, though it lacks persistence and distributed capabilities found in cloud-native alternatives.
- The quality of retrieval is only as good as the quality of chunking. Poorly defined chunk boundaries result in semantically incomplete vectors that fail to match relevant queries.

4.2 Issues Faced

The following issues were encountered during development and subsequently resolved:

- RateLimitError (429): Sending all 39 chunks to the OpenAI embedding API in a single call exceeded the token-per-minute quota. Resolved by implementing batch embedding with inter-batch delays and exception-based retry logic.

- Chunk fragmentation: Initial testing with chunk_size=1000 produced some chunks that split mid-sentence, reducing the semantic coherence of the embedding. Addressed by reducing chunk_size to 300 and tuning the overlap.
- Metadata loss: Early iterations of the pipeline discarded document metadata (filename, page number) during chunking. Fixed by explicitly preserving the metadatas field when constructing the FAISS index.
- Environment dependency conflicts: The project required careful virtual environment management (venv) under Python 3.12 to avoid version conflicts between langchain, langchain_community, and faiss-cpu.

4.3 Possible Improvements

Several enhancements could increase the capability and robustness of the pipeline:

- Persistent vector store: Replace the in-memory FAISS index with a persistent store (e.g., Chroma, Pinecone, or Weaviate) to retain embeddings between sessions without re-computing them.
- Hybrid retrieval: Combine dense vector search (semantic) with sparse keyword search (BM25) using a hybrid retriever to improve recall on domain-specific terminology.
- Re-ranking: Introduce a cross-encoder re-ranker (e.g., Cohere Rerank or a local cross-encoder model) as a post-retrieval step to improve precision of the top-k results.
- Metadata filtering: Enable filtering of retrieval results by document source, page number, or date to support multi-document corpora with heterogeneous content.
- Query expansion: Use the LLM to rewrite or expand the user query before retrieval to improve recall for ambiguous or short queries.
- Evaluation framework: Implement a quantitative evaluation pipeline using metrics such as MRR (Mean Reciprocal Rank), Recall@k, and faithfulness scores (e.g., via RAGAs) to systematically benchmark retrieval quality.
- Async embedding: Replace synchronous batching with asyncio-based concurrent embedding calls (with rate limiting) to significantly reduce total embedding time for larger corpora.

— End of Report —